

第27-28课 PDF解析整理版

来源：飞书「ChatBI 项目系统蓝图」解析整理版。原课件 PDF 附件无下载权限，本文为文档正文导出。

第 27-28课 PDF 解析整理版

第27 + 28课：从“可用”到“可上线”的关键治理层

[要点] 课程总结与关系梳理

这两节课共同构建了企业级ChatBI系统从“可用”到“可上线”的关键治理层。

- 第27课（权限与安全设计）解决了“谁能查、查什么、看什么”的问题。它在SQL执行前后建立了安全闭环，通过行级过滤、列级脱敏和危险SQL拦截，确保数据访问符合企业业务规则，防止越权和数据泄露。这是系统合规性的基础。
- 第28课（SQL错误处理与性能优化）解决了“查得稳、查得快、查得明”的问题。在第27课安全链路的基础上，它进一步处理了执行阶段的工程问题：将模糊的数据库异常翻译为明确的业务错误类型，引入慢查询记录与EXPLAIN采样以提供性能优化证据，并实现连接池复用提升并发能力。这是系统稳定性和可运维性的保障。

总结：

第27课是“安全门”，保证数据不越界；第28课是“稳定器”，保证查询不崩溃且有迹可循。两者结合，才构成一个真正具备企业级服务能力的ChatBI后端。

第 27 课：权限与安全设计：企业数据为什么不能随便查

一、本节目标

1.1 课程定位

第 26 课已经把复杂分析链路接通了，系统可以把一个经营问题拆开、执行多步 SQL，再整理成分析报告。

现在系统离企业可上线状态还差一层治理：同一条查询链路面对不同身份时，返回范围如何限定（必须）？

1.2 当前问题

沿着前面的代码状态，api_service.py 可以直接接收请求，main.py 会生成 SQL，database.py 会执行 SQL。

这条链路已经能查数/分析结果，但实际使用时仍然会有如下 3 个问题：

- 谁可以查
- 能查到哪一层数据
- 返回结果里哪些列要遮挡

企业里的数据系统，不光要“能查出来”，还要保证“查的人对、范围对、结果展示对”。

1.3 本节课目标

- 理解企业数据权限为什么必须落到查询链路里
- 区分行级权限、列级权限、危险 SQL 拦截
- 实现 `security.py`，把权限规则收成独立模块
- 在 `api_service.py`、`main.py`、`database.py` 之间接通权限上下文
- 验证区域过滤、结果脱敏和安全测试是否生效

二、核心思路

2.1 现有链路的风险点

如果继续沿用第 26 课的状态，系统默认把问题直接交给模型，再把 SQL 直接发给数据库。

这会带来 3 类典型风险：

1. 范围越权：华东销售可能查到华南订单
2. 字段越权：普通角色可能看到手机号、邮箱、敏感金额
3. 语句越权：如果模型或人工输入了危险 SQL，数据库层没有兜底

对于企业场景，权限控制最好不要只停在前端按钮是否可见。

前端隐藏只是体验控制，真正能兜底的地方，仍然是请求进入服务之后、SQL 执行之前。

权限控制要落在服务端主链路里，至少要覆盖身份识别、查询约束和结果处理这 3 层。

2.2 本节课的权限模型

2.3 权限链路拆解

本节课的核心处理链路：

这条链路里，每一层的功能：

- `api_service.py`：识别当前用户是谁

- main.py: 把权限上下文带到查询主链路里
- database.py: 把安全规则放到执行前后
- security.py: 集中管理拦截、过滤和脱敏逻辑

2.4 链路中的 3 个安全动作

2.4.1 危险 SQL 拦截

第一步先限制查询类型，只允许 SELECT 和 WITH... SELECT...

目的：

- 阻断 DELETE、UPDATE、DROP 这类修改语句
- 阻断多语句拼接带来的风险
- 给后续审计和告警留下统一出口

2.4.2 行级权限

行级权限解决的是这条记录该不该看。

权限模型要求：销售角色只允许看自己区域的数据，因此要在执行前自动补上区域条件。

例如：

- 原始 SQL: GROUP BY c.region
- 安全改写后: WHERE c.region='华东大区' GROUP BY c.region

2.4.3 列级权限

列级权限解决的是这一列能不能直接展示。

它不改变查询是否成功，只改变结果里的可见内容。

同一批行数据可以允许返回，但列级展示仍然要再过一层权限判断。

三、方案实现

3.1 修改代码

关键代码的作用如下：

3.2 security.py 的关键实现

第一步，将权限规则收进独立模块：

```
@dataclass(slots=True)
class UserContext:
    user_id: str
    role: str = "admin"
    region: str | None = None

class QuerySecurityManager:
    def secure_sql(self, sql: str, user: UserContext | None = None) -> str:
        user_context = user or UserContext.demo_admin()
        normalized_sql = self._normalize_sql(sql)
        self._ensure_select_only(normalized_sql)
        ...

    def mask_result(self, columns: list[str], rows, user: UserContext | None = None):
        ...
```

这里把规则拆成两个入口，刚好对应数据库执行的前后两个阶段：

- `secure_sql()`：执行前调用，负责拦截和行级过滤
- `mask_result()`：执行后调用，负责列级脱敏

3.3 database.py 与 main.py 的接入

数据库执行层先经过 `QuerySecurityManager`，然后才执行 SQL：

```
def execute(self, sql: str, user: UserContext | None = None):
    user_context = user or UserContext.demo_admin()
    secured_sql = self.security.secure_sql(sql, user_context)
    conn = self.connection_factory()
    try:
        with conn.cursor() as cursor:
            cursor.execute(secured_sql)
            columns = [desc[0] for desc in cursor.description] if cursor.description else []
            results = cursor.fetchall()
            _, masked_rows = self.security.mask_result(columns, results, user_context)
            return columns, masked_rows
    finally:
        conn.close()
```

`main.py` 里新增 `security_context` 参数，把角色信息继续往下传：

```
result = self.db.execute(sql, user=user_context)
```

如果权限检查失败，主链路把错误类型收成 `security`，这样 API 层能稳定返回 403。

权限错误要和数据库连接错误、SQL

语法错误分开处理——只有这样，接口层才能给出准确的状态码和提示信息。

3.4 api_service.py 中间件

这一层负责把身份挂到请求上下文里。

```
@app.middleware("http")
async def attach_user_context(request: Request, call_next):
    request.state.user_context = UserContext(
        user_id=request.headers.get("x-user-id", "demo_admin"),
        role=request.headers.get("x-user-role", "admin"),
        region=request.headers.get("x-user-region"),
    )
    return await call_next(request)
```

同步接口和流式接口都会把这个上下文继续传给 ChatBISystem。

这样做的好处：

- 同步接口和 SSE 接口共用一套权限规则
- 查询开关、指标开关、权限上下文都在服务层统一收口
- 后续如果接真实登录系统，只需要替换身份来源，不需要重写数据库安全逻辑

3.5 运行验证

3.5.1 区域过滤改写

下面这条命令直接调用 security.py，验证销售角色会自动补上区域条件：

```
uv run python -c 'from security import QuerySecurityManager, UserContext; manager = QuerySecurityManager(); sql = "SELECT c.region, SUM(o.net_amount) AS revenue FROM sales_orders o JOIN dim_customers c ON o.customer_id = c.customer_id GROUP BY c.region"; user = UserContext(user_id="u_sales_east", role="sales", region="华东大区"); print(manager.secure_sql(sql, user))'
```

实际输出如下：

```
SELECT c.region, SUM(o.net_amount) AS revenue FROM sales_orders o JOIN dim_customers c ON o.customer_id = c.customer_id
WHERE c.region = '华东大区' GROUP BY c.region
```

这条输出说明：

- 行级权限不是停在概念上，而是真的改写了执行前 SQL
- 过滤条件插入在 GROUP BY 之前，保证语句仍然能继续执行

3.5.2 安全测试

本节新增单元测试：

```
uv run pytest tests/test_security.py -q
```

```
..... [100%]
5 passed in 0.45s
```

这 5 个测试覆盖的行为包括：

- DELETE 之类的危险 SQL 会被拦截
- 销售角色查询会自动追加区域过滤
- 手机号和敏感金额会被脱敏
- database.py 会在执行前后接通安全规则
- main.py 会把权限失败归类为 security

到这里，系统已经具备“能查、会限、能挡、会遮”的最小安全闭环。

四、小结

4.1 本节课内容

- 把权限模型从口头约束落成了可执行代码
- 用 security.py 收口了 SQL 拦截、行级过滤和列级脱敏
- 在 api_service.py、main.py、database.py 之间接通了权限上下文
- 用真实命令验证了区域过滤和安全测试

4.2 边界与后续

这一课先完成的是查询安全控制。

第 28 课继续处理：

- 组织架构同步
- 审计日志
- 批准流和审批链
- 慢查询优化
- 连接池和 SQL 执行异常分类

4.3 常见问题

4.3.1 为什么不能只在前端做权限控制

前端隐藏按钮只影响界面，不影响接口是否会被直接调用。

真正可靠的兜底位置，仍然是服务端执行前。

4.3.2 为什么要同时做行级和列级

行级权限决定“哪条记录能返回”，列级权限决定“返回后哪些字段还能展示”。

这两个问题不在同一层，通常都要做。

4.3.3 为什么权限错误要单独归类

因为它对应的是业务授权问题，不是数据库宕机，也不是 SQL 写错。

单独归类后，接口层才能稳定返回 403，排障和审计也更清楚。

五、附录

5.1 代码路径

- code/chatbi_mvp/security.py
- code/chatbi_mvp/database.py
- code/chatbi_mvp/main.py
- code/chatbi_mvp/api_service.py
- code/chatbi_mvp/tests/test_security.py

第 28 课：SQL 错误处理与性能优化

一、本节目标

1.1 课程定位

第 27 课我们接入了权限控制：

- 谁可以查
- 能查哪些行
- 哪些列要脱敏

但 SQL 发到数据库之后，执行层还有一组问题没有处理：

- 出错时，错误类型不够清楚
- 变慢时，如何定位证据——指导后续优化方向

- 查询变多时，连接复用能力不足

因此，本节课处理的是执行阶段的问题：

- 数据库异常怎么分类
- 慢查询信息怎么记录
- 数据库连接怎么复用

1.2 课程目标

核心目标：让系统更可用、更健壮

- 在第 27 课原始代码上复现 3 类执行层问题
- 看清 database.py、main.py、api_service.py 在执行阶段的职责缺口
- 实现最小连接池、数据库异常分类、慢查询记录与 EXPLAIN 采样
- 让 API 层返回更细的错误类型和状态码
- 用真实运行结果验证改造前后差异

二、典型问题分析

2.1 SQL 抛出底层异常

2.1.1 问题表现

执行一段带有语法错误的 SQL 语句：

```
uv run python - <<'PY'
from database import DatabaseClient
client = DatabaseClient()
client.execute("SELECT region SUM(net_amount) FROM sales_orders GROUP BY region")
PY
```

输出如下：

```
Traceback (most recent call last):
...
pymysql.err.ProgrammingError: (1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'SUM(net_amount) FROM sales_orders GROUP BY region' at line 1")
```

2.1.2 根因思考

当前版本的 database.py 结构：

```
conn = self.connection_factory()
with conn.cursor() as cursor:
```

```
cursor.execute(secured_sql)
results = cursor.fetchall()
```

它能执行 SQL，但它不会解释异常（拿到 PyMySQL 的原始异常对象）：

- 业务层不知道这是语法错误
- API 层也不知道该返回什么状态码
- 前端只能看到一段底层报错字符串

执行层需要先把数据库异常翻译成系统自己的错误类型，这样主链路才能继续做提示、重试和告警——不要把任何 IT 侧的错误暴露给用户。

2.2 数据库失败情况无区分

2.2.1 问题表现

首先验证一下主链路的错误归类逻辑：

```
uv run python - <<'PY'
import pymysql
from main import ChatBISystem

class FakeParser:
    def parse(self, question: str) -> str: return question
    def validate(self, parsed: str) -> bool: return True

class FakeLLM:
    def generate_sql(self, _system_msg: str, _prompt: str) -> str: return 'SELECT * FROM sales_orders'

class FakeFormatter:
    def format(self, columns, rows): return ''

class FakeIndicatorKnowledge:
    def get_indicator_context(self, question: str): return {'detected_indicators': [], 'indicator_block': ''}

class FakeDB:
    def execute(self, sql: str, user=None):
        raise pymysql.err.OperationalError(
            2013, 'Lost connection to MySQL server during query (The read operation timed out)'
        )

system = ChatBISystem.__new__(ChatBISystem)
system.parser = FakeParser()
system.llm = FakeLLM()
system.db = FakeDB()
system.formatter = FakeFormatter()
system.indicator_knowledge = FakeIndicatorKnowledge()

print(system.run('查看最近 12 个月订单明细'))
PY
```

真实输出如下：

```
{'success': False, 'sql': 'SELECT * FROM sales_orders', 'error': "(2013, 'Lost connection to MySQL server during query (The read operation timed out)')", 'error_type': 'database', 'metadata': {'detected_indicators': [], 'model': 'gpt-4', 'used_few_shot': True, 'used_rules': True, 'used_guards': True, 'used_indicator_knowledge': True, 'used_schema_linking': False, 'used_indicator_rag': False, 'security_role': 'admin', 'security_region': None}}
```

2.2.2 根因思考

当前 ChatBISystem.run 的执行分支如下：

- 权限异常单独归类为 security (line 179)
- 其他所有数据库失败统一归类为 database (line 198)

这样会导致的问题（用户体验不友好 + 系统上线后没法运营分析）：

- 语法错误和超时被混在一起
- 连接失败和权限拒绝被混在一起
- API 层没有办法区分 422、503、504

这里能看到一个明确现象：系统已经有安全规则，但数据库执行失败还是统一落到 database。

2.3 无慢查询证据，无连接池

2.3.1 问题表现

先验证一下，当前 DatabaseClient 是否有运行时记录和连接池属性：

```
uv run python - <<'PY'
from database import DatabaseClient
client = DatabaseClient()
print({
    'has_last_query_info': hasattr(client, 'last_query_info'),
    'has_connection_pool': hasattr(client, 'connection_pool'),
})
PY
```

真实输出如下：

```
{'has_last_query_info': False, 'has_connection_pool': False}
```

2.3.2 根因思考

无运行时记录和连接池属性，会直接导致：

- 查询结束后，没有地方保存 duration_ms
- 查询变慢后，没有地方补 EXPLAIN

- 每次查询都重新 `pymysql.connect(...)`

这意味着系统虽然能返回结果，但系统/用户完全不知道：

- 慢的是 LLM，还是数据库
- 是哪一张表在全表扫描
- 为什么前一条快，后一条突然慢
- 查询量上来之后，为什么连接成本越来越高

没有执行耗时和查询计划，后续的性能分析无法进行。

三、优化方案

3.1 链路分析

本节课优化点需要涉及 4 个文件：

- `code/chatbi_mvp/config.py`
- `code/chatbi_mvp/database.py`
- `code/chatbi_mvp/main.py`
- `code/chatbi_mvp/api_service.py`

处理链路如下：

3.2 database.py 的改造重点

本节新增了 3 个组件：

关键代码骨架：

```
class QueryExecutionError(RuntimeError):
    def __init__(self, error_type: str, message: str, metadata: dict[str, Any] | None = None):
        ...

class DatabaseConnectionPool:
    def acquire(self) -> Any:
        ...
    def release(self, conn: Any) -> None:
        ...

class DatabaseClient:
    def execute(self, sql: str, user: UserContext | None = None):
        user_context = user or UserContext.demo_admin()
        secured_sql = self.security.secure_sql(sql, user_context)
        started_at = self.time_fn()
        conn = self._acquire_connection()
```

...

同步补充的环境变量：

- DB_CONNECT_TIMEOUT
- DB_READ_TIMEOUT
- DB_WRITE_TIMEOUT
- DB_POOL_SIZE
- DB_POOL_MAX_OVERFLOW
- DB_POOL_TIMEOUT
- DB_SLOW_QUERY_THRESHOLD_MS

3.3 异常分类与慢查询记录

数据库异常现在会被翻译成更明确的类型：

```
if isinstance(exc, pymysql.err.ProgrammingError) or error_code == 1064:
    return QueryExecutionError("sql_syntax", ...)

if error_code in {1044, 1045, 1142, 1143, 1227}:
    return QueryExecutionError("permission_denied", ...)

if error_code in {1205, 3024} or (error_code in {2013} and "timed out" in error_text.lower()):
    return QueryExecutionError("query_timeout", ...)
```

慢查询记录的逻辑放在执行成功之后：

- 记录耗时
- 留下查询计划

```
duration_ms = round((self.time_fn() - started_at) * 1000, 2)
explain_plan = self._explain(cursor, secured_sql, duration_ms)
self.last_query_info = {
    "sql": secured_sql,
    "duration_ms": duration_ms,
    "slow_query": bool(explain_plan),
    "explain_plan": explain_plan,
}
```

3.4 改造后验证

3.4.1 语法错误被识别为 sql_syntax

```
uv run python - <<'PY'
from database import DatabaseClient, QueryExecutionError
client = DatabaseClient()
try:
```

```

    client.execute("SELECT region SUM(net_amount) FROM sales_orders GROUP BY region")
except QueryExecutionError as exc:
    print({
        'error_type': exc.error_type,
        'message': str(exc),
        'code': exc.metadata.get('error_code'),
    })
PY

```

真实输出:

```

{
    'error_type': 'sql_syntax',
    'message': 'SQL 语法错误, 请检查字段、聚合和别名是否正确',
    'code': 1064
}

```

3.4.2 超时被识别为 query_timeout

```

uv run python - <<'PY'
import pymysql
from config import DB_CONFIG
from database import DatabaseClient, QueryExecutionError

cfg = dict(DB_CONFIG)
cfg['read_timeout'] = 1
cfg['write_timeout'] = 1
client = DatabaseClient(connection_factory=lambda: pymysql.connect(**cfg))
try:
    client.execute('SELECT SLEEP(2)')
except QueryExecutionError as exc:
    print({
        'error_type': exc.error_type,
        'message': str(exc),
        'code': exc.metadata.get('error_code'),
    })
PY

```

真实输出:

```

{
    'error_type': 'query_timeout',
    'message': 'SQL 执行超时, 请缩小时间范围或减少返回列后重试',
    'code': 2013
}

```

3.4.3 权限拒绝被识别为 permission_denied

```

uv run python - <<'PY'
import pymysql
from config import DB_CONFIG
from database import DatabaseClient, QueryExecutionError

admin_cfg = dict(DB_CONFIG)

```

```

user_name = 'chatbi_l28_no_select'
password = 'TempL28Pass!'
admin_conn = pymysql.connect(**admin_cfg)
try:
    with admin_conn.cursor() as cursor:
        cursor.execute(f"DROP USER IF EXISTS '{user_name}'@'%'"")
        cursor.execute(f"CREATE USER '{user_name}'@'% ' IDENTIFIED BY '{password}'")

    limited_cfg = dict(DB_CONFIG)
    limited_cfg['user'] = user_name
    limited_cfg['password'] = password
    client = DatabaseClient(connection_factory=lambda: pymysql.connect(**limited_cfg))
    try:
        client.execute('SELECT * FROM sales_orders LIMIT 1')
    except QueryExecutionError as exc:
        print({
            'error_type': exc.error_type,
            'message': str(exc),
            'code': exc.metadata.get('error_code'),
        })
finally:
    with admin_conn.cursor() as cursor:
        cursor.execute(f"DROP USER IF EXISTS '{user_name}'@'%'"")
    admin_conn.close()
PY

```

真实输出:

```

{
    'error_type': 'permission_denied',
    'message': '数据库拒绝执行该查询，请检查当前账号权限',
    'code': 1044
}

```

3.4.4 慢查询会留下 EXPLAIN 证据

将慢查询超时阈值临时调成 1 ms:

```

uv run python - <<'PY'
from database import DatabaseClient
client = DatabaseClient(slow_query_threshold_ms=1)
columns, rows = client.execute(
    "SELECT c.region, SUM(o.net_amount) AS revenue"
    " FROM sales_orders o"
    " JOIN dim_customers c ON o.customer_id = c.customer_id"
    " GROUP BY c.region"
)
print({'columns': columns, 'rows': rows})
print({
    'duration_ms': client.last_query_info['duration_ms'],
    'slow_query': client.last_query_info['slow_query'],
    'explain_plan': [
        {
            'table': row['table'],
            'type': row['type'],

```

```

        'key': row['key'],
        'Extra': row['Extra'],
    }
    for row in client.last_query_info['explain_plan']
],
}))
PY

```

真实输出：

```

Slow query detected: duration_ms=49.28 sql=SELECT c.region, SUM(o.net_amount) AS revenue FROM sales_orders o JOIN
dim_customers c ON o.customer_id = c.customer_id GROUP BY c.region
{'columns': ['region', 'revenue'], 'rows': [(('欧洲', Decimal('24976693.00')), ('北美', Decimal('8319628.46')), ('亚太',
Decimal('11064930.00')))]}
{
    'duration_ms': 49.28,
    'slow_query': True,
    'explain_plan': [
        {'table': 'c', 'type': 'ALL', 'key': None, 'Extra': 'Using temporary'},
        {'table': 'o', 'type': 'ref', 'key': 'customer_id', 'Extra': None}
    ]
}

```

这一段输出中可发现两个字段：

- c 表访问类型是 ALL
- Extra 里出现了 Using temporary

这两项信息可以直接作为后续索引和 SQL 调整的依据——不会优化，也可以将这段 explain 结果发给 AI。

3.5 API 层对齐与回归测试

执行层改完之后，main.py 和 api_service.py 也需要同步调整：

- main.py 把数据库错误细分为 database_sql_syntax、database_query_timeout、database_permission_denied、database_connection_error
- api_service.py 把它们分别映射到 422、504、403、503

完整回归测试：

```
uv run pytest -q
```

输出结果：

```

..... [100%]
29 passed in 7.16s

```

四、小结

到本节课为止，执行层继续补齐了下面几项能力：

- 能识别语法错误
- 能识别超时
- 能识别权限拒绝
- 能记录慢查询
- 能复用数据库连接

下节课内容预告：

- 系统模块如何扩展
- 如何支撑更多、灵活的数据源
- 如何提取出项目核心难点，勾住面试官

附件（飞书无下载权限）：第 27 课：权限与安全设计：企业数据为什么不能随便查.pdf

附件（飞书无下载权限）：第 28 课：SQL 错误处理与性能优化.pdf